

Day 05/05 — Basic POST Endpoint, Validation Lite & Final Demo

This is the final day of the beginner Spring Boot REST foundation.

You already have:

```
GET /books
GET /books/{id}
```

Now you'll add:

```
POST /books
```

The complete flow becomes:

```
HTTP Request
  ↓
Controller
  ↓
Service
  ↓
Repository
  ↓
In-memory List
```

We'll keep this **simple**: Spring Web + in-memory storage. No JPA, database, Spring Security, Bean Validation, or advanced libraries yet.

1. What is POST?

POST is commonly used when the client wants to **create new data**.

For example:

```
POST /books
```

with:

```
{
  "title": "Effective Java",
  "author": "Joshua Bloch"
}
```

The server creates a new book.

Conceptually:

```
Client
  ↓
POST /books
  ↓
Controller
  ↓
Service
  ↓
Repository
  ↓
New Book added
```

2. GET vs POST

HTTP Method	Purpose	Example
GET	Read	GET /books
POST	Create	POST /books

For example:

```
GET /books
```

means:

Give me the books.

While:

POST /books

means:

Create a new book using the data I'm sending.

3. @PostMapping

@PostMapping maps an HTTP POST request to a Java method.

```
@PostMapping("/books")
public String createBook() {
    return "Book created";
}
```

Now:

POST /books

will execute:

createBook()

4. What is @RequestBody?

When a client sends JSON in the HTTP request body:

```
{  
  "title": "Java Basics",  
  "author": "James"  
}
```

you need a way to receive that JSON in Java.

That's what `@RequestBody` does.

Example:

```
@PostMapping("/books")  
public String createBook(@RequestBody BookRequest request) {  
    return request.title();  
}
```

Spring takes the JSON:

```
{  
  "title": "Java Basics",  
  "author": "James"  
}
```

and converts it into:

```
BookRequest
```

Conceptually:

```
JSON  
↓  
Spring/Jackson  
↓  
BookRequest object
```

5. Why Create BookRequest DTO?

Don't directly use your domain/model class for incoming API data.

Create:

```
public record BookRequest(  
    String title,  
    String author  
) {  
}
```

This is a **DTO — Data Transfer Object**.

Its job is to represent data coming from the client.

Request

```
{  
  "title": "Java Basics",  
  "author": "James"  
}
```

becomes:

```
BookRequest(  
    "Java Basics",  
    "James"  
)
```

6. Why Create BookResponse DTO?

Similarly, create a separate DTO for data going **back to the client**.

```
public record BookResponse(  
    int id,  
    String title,  
    String author  
) {  
}
```

So now we have:

```
BookRequest  
  ↓  
Client → Server
```

and:

```
BookResponse  
  ↓  
Server → Client
```

This separation becomes very useful in larger applications.

7. Book Model

Keep your model:

```
public record Book(  
    int id,  
    String title,  
    String author  
) {  
}
```

Now we have three different concepts:

```
Book
  ↓
Internal/domain data

BookRequest
  ↓
Incoming API data

BookResponse
  ↓
Outgoing API data
```

8. Complete Project Structure

Your project can now look like:

```
src/main/java/com/springdemo/demo

├── Controllers
│   └── BookController.java
├── Services
│   └── BookService.java
├── Repositories
│   └── BookRepository.java
├── Models
│   └── Book.java
└── DTOs
    ├── BookRequest.java
    └── BookResponse.java
```

Package names can differ; the important thing is the separation of responsibilities.

9. BookRequest

```
package com.springdemo.demo.DTOS;  
  
public record BookRequest(  
    String title,  
    String author  
) {  
}
```

This represents what the client sends.

Example:

```
{  
  "title": "Spring Boot",  
  "author": "Robert"  
}
```

10. BookResponse

```
package com.springdemo.demo.DTOS;  
  
public record BookResponse(  
    int id,  
    String title,  
    String author  
) {  
}
```

This represents what the API sends back.

Example:


```
{
  "id": 3,
  "title": "Spring Boot",
  "author": "Robert"
}
```

11. Update Repository

Our repository currently has:

```
@Repository
public class BookRepository {

    private final List books = new ArrayList<>(
        List.of(
            new Book(1, "Java Basics", "James"),
            new Book(2, "Spring Boot", "Robert")
        )
    );

    public List findAll() {
        return books;
    }

    public Optional findById(int id) {
        return books.stream()
            .filter(book -> book.id() == id)
            .findFirst();
    }
}
```

We now need a method to add a book.

```
public Book save(Book book) {
    books.add(book);
    return book;
}
```

So the complete repository becomes:

```
package com.springdemo.demo.Repositories;

import com.springdemo.demo.Models.Book;
import org.springframework.stereotype.Repository;

import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

@Repository
public class BookRepository {

    private final List books = new ArrayList<>(
        List.of(
            new Book(1, "Java Basics", "James"),
            new Book(2, "Spring Boot", "Robert")
        )
    );

    public List findAll() {
        return books;
    }

    public Optional findById(int id) {
        return books.stream()
            .filter(book -> book.id() == id)
            .findFirst();
    }

    public Book save(Book book) {
        books.add(book);
        return book;
    }
}
```

12. Why ArrayList?

Earlier we used:

```
List.of(...)
```

But List.of() creates an **unmodifiable list**.

We need to add new books:

```
books.add(book);
```

Therefore, we use:

```
new ArrayList<>(List.of(...))
```

Now the list can be modified.

This is an important Java detail.

13. Update BookService

The service receives the BookRequest.

```
@Service
public class BookService {

    private final BookRepository bookRepository;

    public BookService(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    public List getAllBooks() {
        return bookRepository.findAll();
    }

    public Optional getBookById(int id) {
        return bookRepository.findById(id);
    }
}
```

```

public BookResponse createBook(BookRequest request) {

    int id = bookRepository.findAll().size() + 1;

    Book book = new Book(
        id,
        request.title(),
        request.author()
    );

    Book savedBook = bookRepository.save(book);

    return new BookResponse(
        savedBook.id(),
        savedBook.title(),
        savedBook.author()
    );
}
}

```

Notice the flow:

```

BookRequest
  ↓
Service
  ↓
Book
  ↓
Repository
  ↓
Book
  ↓
BookResponse

```

14. Add Simple Manual Validation

The requirement says:

Validate missing title and author.

We can keep this extremely simple.

```
public BookResponse createBook(BookRequest request) {

    if (request.title() == null || request.title().isBlank()) {
        throw new IllegalArgumentException("Title is required");
    }

    if (request.author() == null || request.author().isBlank()) {
        throw new IllegalArgumentException("Author is required");
    }

    int id = bookRepository.findAll().size() + 1;

    Book book = new Book(
        id,
        request.title(),
        request.author()
    );

    Book savedBook = bookRepository.save(book);

    return new BookResponse(
        savedBook.id(),
        savedBook.title(),
        savedBook.author()
    );
}
```

For this beginner exercise, this is enough.

Later you'll learn:

```
Jakarta Bean Validation
@Valid
@NotBlank
@Size
@Min
@Max
```

Don't introduce those yet.

15. Add POST to Controller

```
@PostMapping
public BookResponse createBook(@RequestBody BookRequest request) {
    return bookService.createBook(request);
}
```

Because our controller already has:

```
@RequestMapping("/books")
```

the complete endpoint is:

```
POST /books
```

16. Complete BookController

Here's the complete beginner controller:

```
package com.springdemo.demo.Controllers;

import com.springdemo.demo.DTOS.BookRequest;
import com.springdemo.demo.DTOS.BookResponse;
import com.springdemo.demo.Models.Book;
import com.springdemo.demo.Services.BookService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/books")
public class BookController {

    private final BookService bookService;
```

```

public BookController(BookService bookService) {
    this.bookService = bookService;
}

@GetMapping
public List getBooks() {
    return bookService.getAllBooks();
}

@GetMapping("/{id}")
public ResponseEntity getBookById(@PathVariable int id) {
    return bookService.getBookById(id)
        .map(ResponseEntity::ok)
        .orElseGet(() -> ResponseEntity.notFound().build());
}

@PostMapping
public BookResponse createBook(@RequestBody BookRequest request) {
    return bookService.createBook(request);
}
}

```

This gives you:

```

GET    /books
GET    /books/{id}
POST   /books

```

17. POST Request in Postman

Select:

POST

URL:

```
http://localhost:8080/books
```

Go to:

```
Body → raw → JSON
```

Send:

```
{  
  "title": "Effective Java",  
  "author": "Joshua Bloch"  
}
```

18. What Happens Internally?

This is the most important part.

The client sends:

```
{  
  "title": "Effective Java",  
  "author": "Joshua Bloch"  
}
```

Step 1 — Controller

```
@PostMapping  
public BookResponse createBook(@RequestBody BookRequest request)
```

Spring converts JSON into:

```
BookRequest
```

Step 2 — Controller calls Service

```
return bookService.createBook(request);
```

Step 3 — Service validates

```
if (request.title() == null || request.title().isBlank())
```

and:

```
if (request.author() == null || request.author().isBlank())
```

Step 4 — Service creates Book

```
Book book = new Book(  
    id,  
    request.title(),  
    request.author()  
);
```

Step 5 — Service calls Repository

```
bookRepository.save(book);
```

Step 6 — Repository adds it

```
books.add(book);
```

Step 7 — Service creates response DTO

```
new BookResponse(  
    savedBook.id(),  
    savedBook.title(),  
    savedBook.author()  
);
```

Step 8 — Spring converts response to JSON

```
BookResponse  
    ↓  
Jackson  
    ↓  
JSON
```

Client receives:

```
{  
  "id": 3,  
  "title": "Effective Java",  
  "author": "Joshua Bloch"  
}
```

19. Complete POST Flow

Memorize this:

```
JSON Request  
    ↓  
@RequestBody  
    ↓  
BookRequest  
    ↓
```

```
Controller
  ↓
Service
  ↓
Validation
  ↓
Book Model
  ↓
Repository
  ↓
In-memory List
  ↓
BookResponse
  ↓
JSON Response
```

20. Missing Title

Suppose Postman sends:

```
{
  "title": "",
  "author": "Joshua Bloch"
}
```

Service detects:

```
request.title().isBlank()
```

and throws:

```
throw new IllegalArgumentException("Title is required");
```

For this beginner exercise, you'll see an error response from Spring.

Later, you'll learn how to create a proper standardized error response and return 400 Bad Request.

21. Missing Author

Request:

```
{
  "title": "Effective Java",
  "author": ""
}
```

Service detects the invalid author and throws:

```
throw new IllegalArgumentException("Author is required");
```

Again, proper exception handling will come later.

22. 200 vs 201

For successful GET:

```
200 OK
```

means:

■ The request was successfully processed.

For POST creating a new resource, REST APIs commonly use:

```
201 Created
```

meaning:

A new resource was successfully created.

For today's exercise, you don't need to implement `ResponseEntity` for this.

Just understand:

```
GET successful → 200 OK  
POST creates resource → commonly 201 Created
```

23. Why Not Always Return 200?

Because HTTP status codes communicate what happened.

For example:

```
200 → Request succeeded  
201 → Resource created  
400 → Client sent invalid request  
404 → Resource not found  
500 → Server-side error
```

You'll work with these more deeply later.

24. Important Correction About `ResponseEntity`

Since you caught the problem in the previous lesson, here's the clean version.

For GET by ID:

```
@GetMapping("/{id}")  
public ResponseEntity getBookById(@PathVariable int id) {
```

```
return bookService.getBookById(id)
    .map(ResponseEntity::ok)
    .orElseGet(() -> ResponseEntity.notFound().build());
}
```

This is type-safe because both branches produce:

```
ResponseEntity
```

The not-found response contains no body:

```
ResponseEntity.notFound().build()
```

For today's POST exercise, we don't need `ResponseEntity`:

```
@PostMapping
public BookResponse createBook(@RequestBody BookRequest request) {
    return bookService.createBook(request);
}
```

Keep it simple.

25. DTO Mapping

Notice that we don't return the request directly.

Request:

```
BookRequest
```

becomes:

```
Book
```

and then:

```
BookResponse
```

Conceptually:

```
BookRequest
  ↓
  Service
  ↓
  Book
  ↓
BookResponse
```

This separation becomes very important when APIs become larger.

26. Why Not Use Book as Request?

You could technically do:

```
@PostMapping
public Book createBook(@RequestBody Book book) {
    ...
}
```

But that's not what we're practicing.

Using:

```
BookRequest
```

and:

```
BookResponse
```

teaches an important professional concept:

API contracts should be separated from internal/domain models.

For example, your internal Book could later have fields that you don't want clients to send or receive.

27. Why Is the ID Not in BookRequest?

The client sends:

```
{
  "title": "Effective Java",
  "author": "Joshua Bloch"
}
```

It doesn't send:

```
{
  "id": 3,
  "title": "Effective Java",
  "author": "Joshua Bloch"
}
```

because the server is responsible for assigning the ID in our simple example.

So:

```
BookRequest
  ↓
title
author
```

while:


```
BookResponse
```

```
↓
```

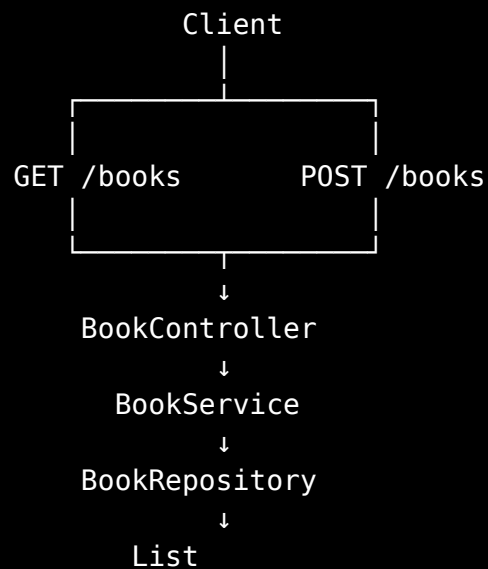
```
id
```

```
title
```

```
author
```

28. Final Project Flow

Your complete beginner application now looks like:



For POST:

```
JSON
```

```
↓
```

```
BookRequest
```

```
↓
```

```
Controller
```

```
↓
```

```
Service
```

```
↓
```

```
Validation
```

```
↓
```

```
Book
↓
Repository
↓
List
↓
BookResponse
↓
JSON
```

29. Final Demo

1. Start application

Run:

```
DemoApplication
```

Spring should start successfully.

2. GET all books

```
GET http://localhost:8080/books
```

Initially:

```
[
  {
    "id": 1,
    "title": "Java Basics",
    "author": "James"
  },
  {
    "id": 2,
    "title": "Spring Boot",
    "author": "Robert"
  }
]
```

```
}  
]
```

3. GET one book

```
GET http://localhost:8080/books/1
```

Response:

```
{  
  "id": 1,  
  "title": "Java Basics",  
  "author": "James"  
}
```

4. GET non-existing book

```
GET http://localhost:8080/books/99
```

Response:

```
404 Not Found
```

5. POST new book

```
POST http://localhost:8080/books
```

Body:

```
{
  "title": "Effective Java",
  "author": "Joshua Bloch"
}
```

Response:

```
{
  "id": 3,
  "title": "Effective Java",
  "author": "Joshua Bloch"
}
```

6. GET all books again

```
GET http://localhost:8080/books
```

Now:

```
[
  {
    "id": 1,
    "title": "Java Basics",
    "author": "James"
  },
  {
    "id": 2,
    "title": "Spring Boot",
    "author": "Robert"
  },
  {
    "id": 3,
    "title": "Effective Java",
    "author": "Joshua Bloch"
  }
]
```

This proves your POST actually modified the in-memory list.

30. Important Limitation of In-Memory Storage

If you stop the Spring Boot application:

```
Application stopped
↓
Memory cleared
```

When you start it again:

```
Application starts
↓
List recreated
↓
Only initial books exist
```

The new book is gone.

That's because we're not using a database yet.

Later:

```
Repository
↓
Database
```

will provide persistent storage.

31. Common Mistakes

Mistake 1 — Forgetting @RequestBody

Wrong:

```
public BookResponse createBook(BookRequest request)
```

Correct:

```
public BookResponse createBook(@RequestBody BookRequest request)
```

Mistake 2 — Wrong Content-Type

In Postman, use:

```
Body → raw → JSON
```

Spring expects JSON.

Mistake 3 — Putting POST logic in Controller

Don't do:

```
@PostMapping
public BookResponse createBook(...) {

    // validation

    // ID generation

    // creating Book

    // storing Book

    // mapping response

}
```

Keep the controller thin:

```
@PostMapping
public BookResponse createBook(@RequestBody BookRequest request) {
    return bookService.createBook(request);
}
```

Mistake 4 — Using List.of() for a mutable repository

This:

```
private final List books = List.of(...);
```

cannot be modified with:

```
books.add(book);
```

Use:

```
new ArrayList<>(List.of(...))
```

32. Interview Questions

Q1. What does @PostMapping do?

It maps an HTTP POST request to a controller method.

Q2. What does @RequestBody do?

It tells Spring to read the HTTP request body and convert it into a Java object.

Q3. What is a DTO?

A DTO, or Data Transfer Object, is an object used to transfer data between parts of an application or across an API boundary.

Q4. Why use BookRequest instead of Book?

It separates incoming API data from the internal/domain model and gives us control over what clients are allowed to send.

Q5. Why use BookResponse?

It defines what data the API exposes to the client and keeps the response contract separate from internal models.

Q6. What happens when a POST request arrives?

Strong answer:

Spring maps the request to a controller method using `@PostMapping`. `@RequestBody` tells Spring to deserialize the JSON request body into a `BookRequest`. The controller passes that request to the service. The service validates it, creates a `Book`, and asks the repository to store it. The service then creates a `BookResponse`, which Spring serializes into JSON for the client.

Q7. Why is validation in the service in our example?

Because validation here represents application/business rules, and we want to keep the controller focused on HTTP handling.

Later, request-shape validation can also be handled declaratively using Bean Validation.

Q8. What is the difference between 200 and 201?

200 OK means the request succeeded. 201 Created specifically indicates that a new resource was successfully created.

Q9. Why does the new book disappear after restarting the application?

Because the repository stores books only in memory. The list is recreated when the application starts.

Q10. Explain the complete architecture.

The Controller is the HTTP boundary. It receives the request and delegates to the Service. The Service contains business/application logic and calls the Repository. The Repository handles data access. In our current project, the repository uses an in-memory list. The result then travels back through the layers and Spring converts the response object to JSON.

33. Final 5-Day Foundation

You have now built the basic foundation:

```
DAY 1
Spring Boot basics
  ↓
DAY 2
Dependency Injection
  ↓
DAY 3
Controller + Service + Repository
  ↓
DAY 4
GET REST API
  ↓
DAY 5
POST + DTO + basic validation
```

Your current application understands:

```
Spring Boot
├── Beans
├── Dependency Injection
├── Controllers
├── Services
├── Repositories
├── GET
├── POST
├── Path Variables
├── Request Parameters
├── Request Body
```

- └─ DTOs
- └─ JSON
- └─ Basic validation

34. What Comes Next?

After this foundation, the natural progression is:

1. Spring Data JPA
- ↓
2. Database + PostgreSQL/MySQL
- ↓
3. Proper Bean Validation
- ↓
4. Exception Handling
- ↓
5. DTO Mapping
- ↓
6. Spring Security
- ↓
7. JWT Authentication
- ↓
8. Unit Testing
- ↓
9. Mockito
- ↓
10. Integration Testing
- ↓
11. Swagger/OpenAPI
- ↓
12. Docker
- ↓
13. CI/CD
- ↓
14. Deployment
- ↓
15. Advanced Spring Boot

For your current level, **don't jump to Security, JWT, Microservices, or Docker yet**. First become completely comfortable with:

Controller → Service → Repository

and:

GET → POST → RequestBody → DTO → JSON

That foundation will make the next Spring topics much easier.